

Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота № 2

з дисципліни «Розробка мобільних застосунків
під Android»

Тема: «Дослідити створення та взаємодію з компонентом Фрагмент
(Fragment) компоненту Діяльність та набути практичні навички з
використання
фрагментів для інтерфейсу користувача.»

Виконав:
студент групи ІА-31
Саkun Дмитро

Перевірів:
Орленко С.П

Київ 2026

Мета роботи: дослідити створення та взаємодію з компонентом Фрагмент (Fragment) компоненту Діяльність та набуті практичні навички з використання фрагментів для інтерфейсу користувача.

ЗАВДАННЯ

Написати програму під платформу Андроїд, яка має інтерфейс, побудований з декількох фрагментів згідно варіанту. Перший фрагмент представляє з себе форму для введення даних та кнопку підтвердження («ОК»), а інший фрагмент відображає результат взаємодії. Тобто другий фрагмент містить тестове поле з результатом та кнопкою «Cancel» (якщо згідно варіанту така існує, якщо ж за варіантом її немає – можете додати за власним бажанням), яка очищає або приховує (або видаляє) другий фрагмент та очищає форму введення з першого фрагменту. Зверніть увагу, що робота з фрагментами відбувається в рамках однієї Діяльності.

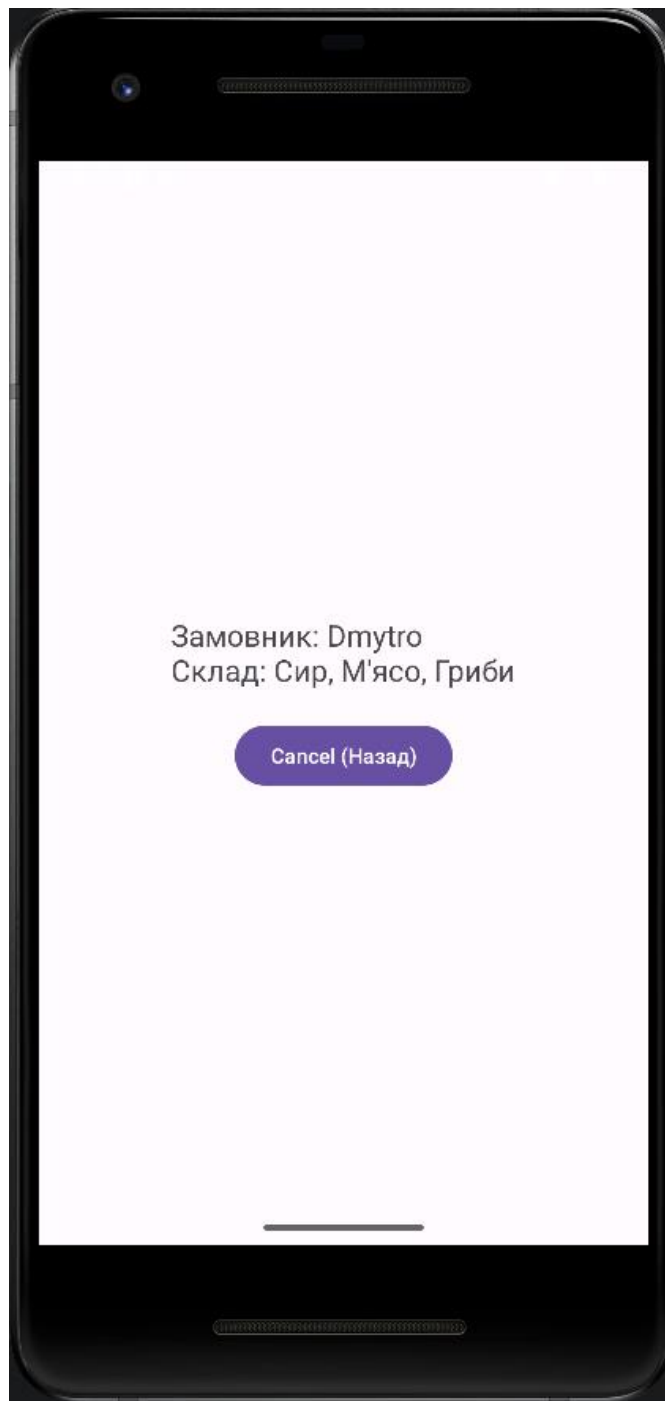
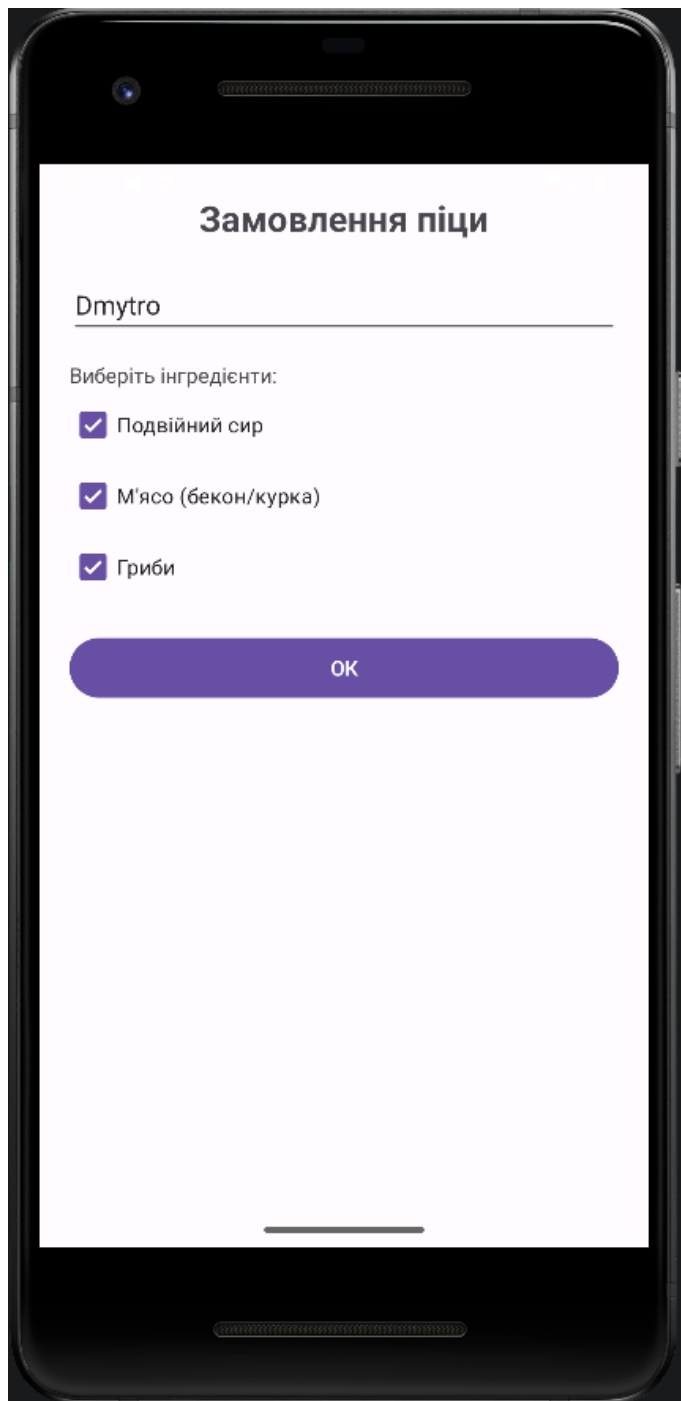
Хід роботи

Варіант	<i>Вікно інтерфейсу</i>
7.	Вікно замовлення піци містить: текстове поле, групи прапорів (тип, розмір тощо), тобто чек-бокси, та кнопку «ОК». Вивести інформацію щодо замовлення при натисканні на кнопку «ОК» у деяке текстове поле.

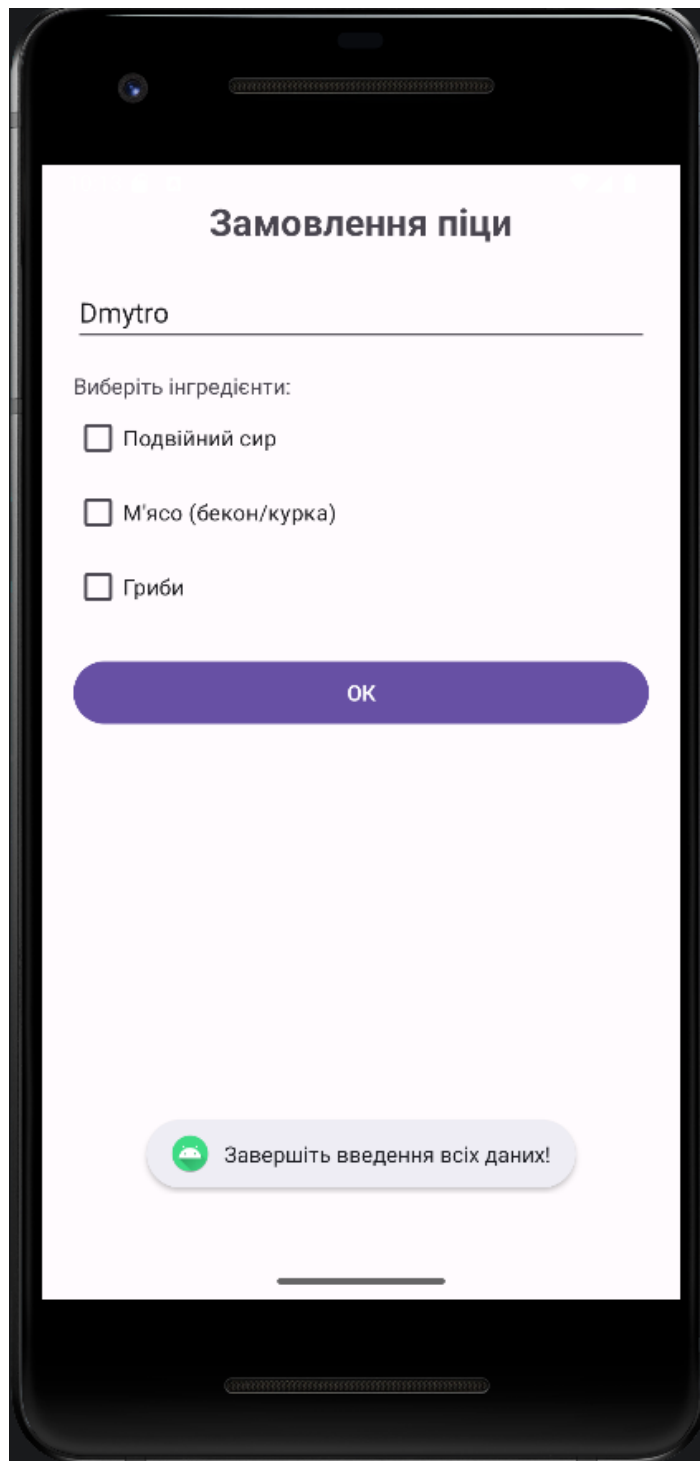
Опис реалізації

- **Головний контейнер:** Використано `FrameLayout` в `activity_main.xml` для динамічної заміни фрагментів.
- **Перший фрагмент (InputFragment):** Реалізовано форму замовлення піци з валідацією імені та вибору інгредієнтів.
- **Другий фрагмент (ResultFragment):** Відображає підсумкову інформацію.
- **Взаємодія:** Для передачі даних використано `Shared ViewModel (PizzaViewModel)`, що дозволяє синхронізувати стан між екранами.

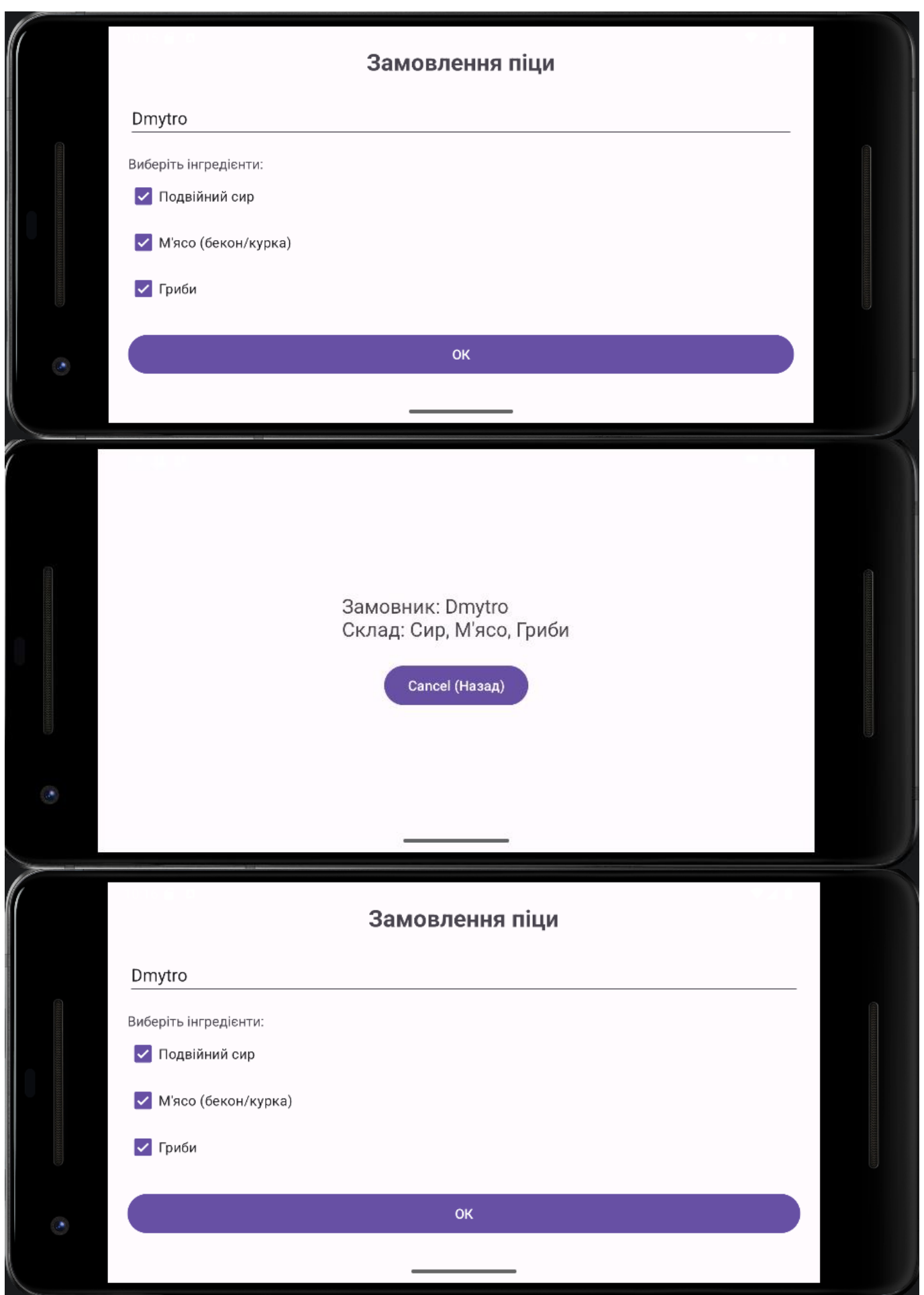
Результати роботи



Екран №1: Форма з введеним ім'ям та обраними інгредієнтами



Екран №2: Повідомлення Toast, якщо натиснув "OK", не вибравши інгредієнти



Екран №3: Перевірка збереження стану даних при зміні конфігурації пристрою (поворот екрана на 90°) за допомогою компонента ViewModel.

Висновок

У ході виконання лабораторної роботи №2 було вивчено принципи роботи з компонентом **Fragment** у середовищі Android Studio на мові Kotlin.

Основні досягнення:

- **Модульність:** Інтерфейс застосунку розділено на два функціональні блоки: `InputFragment` (введення даних замовлення) та `ResultFragment` (відображення результату).
- **Управління станом:** Використано **Shared ViewModel** (`PizzaViewModel`) для обміну даними між фрагментами. Це дозволило реалізувати стабільне збереження введеної інформації (ім'я та вибрані інгредієнти) навіть при зміні конфігурації пристрою, наприклад, при повороті екрана.
- **Навігація:** Опановано роботу з `FragmentManager` та `FragmentTransaction` для динамічної заміни фрагментів у контейнері `FrameLayout`.
- **Валідація:** Реалізовано логіку перевірки заповнення полів перед переходом до наступного екрана, що забезпечує надійність роботи програми.

Відповіді на контрольні питання

1. Призначення та можливості компоненту Фрагмент.

Фрагмент — це модульна частина інтерфейсу користувача в `Activity`. Він дозволяє розділити код на логічні частини, які можна повторно використовувати або комбінувати залежно від розміру екрана пристрою.

2. Життєвий цикл компонента Фрагмент.

Основні етапи:

- `onAttach()` — зв'язок з `Activity`.
- `onCreate()` — ініціалізація компонентів.
- `onCreateView()` — створення ієрархії `View` фрагмента.
- `onViewCreated()` — налаштування логіки елементів керування.
- `onStart()`, `onResume()` — фрагмент стає видимим та активним.
- `onPause()`, `onStop()` — фрагмент іде у фоновий режим.
- `onDestroyView()`, `onDetach()` — завершення роботи та розрив зв'язку з `Activity`.

3. Способи створення компонента Фрагмент.

- **Статично:** через XML-файл розмітки `Activity` за допомогою тегу `<fragment>` або `FragmentManager`.
- **Динамічно (як у цій роботі):** програмно через код за допомогою `FragmentManager` та транзакцій (`beginTransaction()`).

4. Управління компонентом Фрагмент.

Здійснюється через `FragmentManager`. Основні операції:

- `add()` — додати фрагмент.

- `replace()` — замінити існуючий фрагмент новим.
- `remove()` — видалити фрагмент.
- `addToBackStack()` — додати транзакцію до стека, щоб користувач міг повернутися назад.

5. Взаємодія між Фрагментами.

- **Shared ViewModel (найкращий метод):** фрагменти підписуються на одну й ту саму ViewModel, прив'язану до Activity.
- **Fragment Result API:** передача даних через систему ключ-значення.
- **Interfaces:** створення інтерфейсу-коллбека в Activity (застарілий метод).

6. Поняття системи, малої системи та мобільної платформи.

- **Система** — сукупність взаємопов'язаних елементів.
- **Мала система** — підмножина великої системи з чітко обмеженим функціоналом.
- **Мобільна платформа** — програмно-апаратне середовище (ОС + API + SDK), призначене для роботи мобільних пристроїв.

7. Типи мобільних застосунків.

- **Нативні:** розроблені під конкретну ОС (Android на Kotlin, iOS на Swift). Мають найвищу швидкість роботи.
- **Веб-застосунки:** сайти, адаптовані під мобільний браузер.
- **Гібридні:** поєднують нативний контейнер та веб-технології (наприклад, Flutter, React Native).

8. Середовища розробки мобільних застосунків (IDE).

- **Android Studio:** офіційна IDE від Google для Android.
- **Xcode:** для розробки під iOS.
- **VS Code:** часто використовується для кросплатформової розробки. Характеризуються наявністю емуляторів, дебагерів та візуальних редакторів розмітки.

9. Класифікація мобільних платформ.

- **За відкритістю:** відкриті (Android) та закриті (iOS).
- **За типом ліцензії:** пропрієтарні та вільні.